

SIMATS ENGINEERING (SAVEETHA SCHOOL OF ENGINEERING)

Department of Artificial Intelligence and Data Science

ASSESSMENT TOOL 2 – Case Study

Course Code:	DSA03	Course Title:	Natural Language Processing
CO Assessed:	CO3 – Precision	Assessment:	Assessment Tool 2 – Case Study
Weightage:	30% of CIA		
CO5 Statement:	Apply N-gram models, smoothing techniques, part-of-speech tagging systems and to evaluate effective syntactic analysis. (BL-3)		
Student Name:	B.VANI	Reg. No.:	192472348
Section / Batch:	CSE-AI/2024	Date:	

Code of Conduct

I B.VANI(192472348) (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the Student: B.VANI

Instructions

- Read all three case studies carefully before attempting the questions.
- Provide appropriate justifications and interpretations for every computed result.
- Use NLP concepts, algorithms, and terminology wherever applicable.
- Diagrams, flowcharts, tables, or mathematical formulations may be used to support your answers.
- Duration: 30 minutes.

Section / Topic	Focus	Case Study
N-Gram Language Models and Smoothing Techniques	Bigram and Trigram probability computation, Maximum Likelihood Estimation (MLE), Backoff models, Deleted Interpolation, Entropy calculation, uncertainty analysis, real-time next-word prediction	Case Study 1 – Smart Mobile Keyboard Prediction System
Part-of-Speech (POS) Tagging and Word Classes	Penn Treebank tagset, contextual ambiguity resolution, rule-based tagging, stochastic tagging (HMM), emission and transition probabilities, word class identification, chatbot language understanding	Case Study 2 – AI-Powered Customer Support Chatbot
Transformation-Based Tagging (Brill Tagger)	POS tag correction, transformation rules, contextual tagging, linguistic validation, tag sequence refinement, ambiguity handling	Case Study 3 – News Analytics and POS Tag Correction System
Corpus Statistics and Probabilistic Analysis	Word frequency counting, corpus-based probability estimation, entropy-based uncertainty measurement, tagging confidence evaluation	Case Study 3 – News Analytics and POS Tag Correction System
Integrated Syntactic Analysis and NLP Applications	Application of language models, POS tagging, probabilistic methods, corpus analysis, ambiguity resolution, and decision-making in real-world NLP systems	Case Studies 1, 2, and 3

CASE STUDY 1: E-Commerce Smart Search and Product Prediction System

An e-commerce company is developing an AI-powered search and autocomplete system that predicts the next word when customers enter product-related queries. The language model is trained on the corpus:

"machine learning improves business, machine learning enables automation, machine learning drives innovation."

The development team uses **Bigram and Trigram Language Models, Backoff Models, Deleted Interpolation, and Entropy Analysis** to improve search prediction and handle unseen word sequences.

The following statistics are obtained from the corpus:

- $C(\text{machine}) = 3$
- $C(\text{machine learning}) = 3$
- $C(\text{learning improves}) = 1$
- $C(\text{learning enables}) = 1$
- $C(\text{learning drives}) = 1$
- $C(\text{learning improves business}) = 1$

The interpolation weights are:

- $\lambda_1 = 0.5$ (Trigram)
- $\lambda_2 = 0.3$ (Bigram)
- $\lambda_3 = 0.2$ (Unigram)

The prediction probabilities after the word "**learning**" are:

- $P(\text{improves}) = 0.33$
- $P(\text{enables}) = 0.33$
- $P(\text{drives}) = 0.33$

Questions

1. Compute the Maximum Likelihood Estimate (MLE) of the Bigram Probability **$P(\text{learning} | \text{machine})$** . Interpret how this probability can influence the next-word prediction of an e-commerce search system.
2. A customer enters the unseen sequence "**machine learning transforms**". Apply the Backoff Model and estimate the probability of the word "**transforms**" using lower-order N-grams. Explain why backoff is useful when customers enter previously unseen search queries.
3. Using Deleted Interpolation, calculate the probability of the sequence "**machine learning improves**" using the given interpolation weights. Explain how combining trigram, bigram, and unigram probabilities improves robustness when training data is sparse.
4. Calculate the Entropy of the prediction distribution **$P(\text{improves}) = 0.33$, $P(\text{enables}) = 0.33$, $P(\text{drives}) = 0.33$** . Interpret the entropy value and explain what it indicates about uncertainty in the search prediction system.

5. Develop a Python program using **NLTK** for N-gram generation, **NumPy** for probability computations, and the **Math** library for entropy calculation. The program should construct Unigram, Bigram, and Trigram models from the given corpus, compute the MLE Bigram probability, estimate an unseen sequence using Backoff, calculate the Deleted Interpolation probability using the given weights, compute entropy of the prediction distribution, and display all intermediate calculations and the final next-word prediction in a structured format.

CASE STUDY 2: AI-Powered Hospital Appointment Chatbot

A healthcare organization develops an AI-powered chatbot that assists patients in booking appointments. The chatbot performs **Part-of-Speech tagging** before identifying user intentions and generating responses.

Consider the following user inputs:

1. **"Book an appointment with the doctor."**
2. **"The book contains medical information."**

The chatbot uses the **Penn Treebank POS Tagset** and an **HMM-based probabilistic POS tagger** to determine the grammatical role of ambiguous words.

For the word **"book"**, the system is given:

- $P(\text{book} \mid \text{VB}) = 0.7$
- $P(\text{book} \mid \text{NN}) = 0.3$
- $P(\text{Start} \rightarrow \text{VB}) = 0.6$
- $P(\text{Start} \rightarrow \text{NN}) = 0.4$

The system must determine whether **"book"** functions as a Verb (VB) or Noun (NN) depending on its context.

Questions

1. Assign appropriate **Penn Treebank POS tags** to every word in both sentences. Justify the POS tag assigned to **"book"** using the grammatical context of each sentence.
2. Using the given HMM probabilities, compute the likelihood of **"book"** being tagged as a Verb (VB) and as a Noun (NN). Determine the most probable tag and explain why the probabilistic model favors that tag at the beginning of a sentence.
3. Compare **Rule-Based POS Tagging** and **Stochastic HMM Tagging** for resolving the ambiguity of the word **"book"**. Discuss the advantages and limitations of both approaches in a healthcare chatbot.
4. Evaluate how standardized POS tagsets such as the **Penn Treebank Tagset** can support downstream NLP tasks such as **intent detection**, **entity identification**, and **response generation** in healthcare chatbots.
5. Develop a Python program using **NLTK** to tokenize the given sentences and assign Penn Treebank POS tags. Implement a simple **HMM-based POS tagging calculation** using the given emission and transition probabilities. The program should calculate the probability of **"book"** being tagged as VB and NN, determine the most likely tag, display the tokenized sentences, POS-tagged output, intermediate probability calculations, and final tagging results in a structured format.

CASE STUDY 3: Financial News POS Tag Correction and Uncertainty Analysis

A financial news analytics company processes thousands of financial news articles every day. The system initially assigns POS tags using a statistical POS tagger and subsequently applies a **Transformation-Based Tagger (Brill Tagger)** to correct common tagging errors.

The sentence processed by the system is:

"Market growth drives investment."

The initial POS tags produced by the system are:

- market/NN
- growth/NN
- drives/NNS
- investment/NN

A learned transformation rule states:

Change NNS to VBZ if the preceding word is tagged as NN.

The system also performs corpus-based frequency analysis and entropy-based uncertainty evaluation.

The corpus contains the following word frequencies:

1. market – 500 occurrences
2. growth – 350 occurrences
3. drives – 180 occurrences
4. investment – 420 occurrences

Questions

1. Apply the given transformation rule to the initial POS-tag sequence. Display the corrected POS-tag sequence and explain why the transformation is linguistically appropriate.
2. Analyze the initial tagging error for **"drives"**. Explain why the word should receive the **VBZ** tag in this sentence and discuss how the surrounding grammatical context helps identify the correct tag.
3. Using the given corpus statistics, calculate the **relative frequency/probability** of each word. Explain how corpus frequency information can support probabilistic POS tagging and language processing.
4. Calculate the **entropy of the word-frequency distribution** before and after applying the transformation rule. Explain what entropy indicates about uncertainty in the tagging or prediction process.
5. Develop a Python program using **NLTK** for initial POS tagging and implement a simple **Transformation-Based Tagger** that applies the rule **"Change NNS to VBZ if the preceding word is tagged as NN."** The program should tokenize the sentence, generate initial POS tags, apply the transformation rule, calculate word-frequency probabilities from the given corpus statistics, calculate entropy using the Python **math** library, and display the initial tags, corrected tags, frequency distribution, entropy values, and final POS-tagging results in a structured format.

Rubrics for Calculation

Criteria	Weightage	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Understanding of Case	25	Thorough understanding; clearly identifies all key issues	Good understanding with most issues identified	Basic understanding; some issues missed	Poor understanding; problem not identified correctly
Application of Concepts	25	Effectively applies relevant concepts to analyze the case deeply	Applies appropriate concepts with minor gaps	Limited application of concepts	Fails to apply relevant concepts
Analysis	20	In-depth analysis with strong reasoning and insights	Good analysis with logical reasoning	Basic analysis with limited depth	Weak or no analysis; lacks reasoning
Solution Design	20	Innovative, feasible solutions with strong justification	Logical solutions with adequate justification	Basic solutions with weak justification	Incorrect or no solution provided
Clarity	10	Clear, well-structured, and easy to understand	Organized and understandable presentation	Limited clarity and structure	Poor clarity; difficult to understand

Q. No. / Criteria Level	Understanding of Case	Application of Concepts	Analysis	Solution Design	Clarity	Sub. Total	Total %
1							
2							
3							

Faculty In-charge	Course Coordinator	Program Director
Signature: _____	Signature: _____	Signature: _____
Date: _____	Date: _____	Date: _____

DSA0306-NATURAL LANGUAGE PROCESSING

CO3 AT 2

B.VANI(192472348)

Q1. Compute the MLE of the Bigram Probability $P(\text{learning} \mid \text{machine})$.

Given:

$$C(\text{machine}) = 3$$

$$C(\text{machine learning}) = 3$$

Formula:

$$P(\text{learning} \mid \text{machine}) = C(\text{machine learning}) / C(\text{machine})$$

$$P(\text{learning} \mid \text{machine}) = 3 / 3$$

$$P(\text{learning} \mid \text{machine}) = 1.0$$

Therefore, the MLE of the bigram probability is 1.0 or 100%.

Interpretation:

This means that in the given corpus, every occurrence of the word "machine" is followed by "learning". Therefore, when a customer types "machine", the search system assigns a very high probability to "learning" as the next word. This improves autocomplete prediction for product-related queries.

Q2. Apply the Backoff Model for the unseen sequence "machine learning transforms". The required trigram is:

(machine, learning, transforms)

This trigram is unseen in the corpus.

Step 1: Check Trigram:

$P(\text{transforms} \mid \text{machine learning}) = 0$

because $C(\text{machine learning transforms}) = 0$.

Step 2: Back off to Bigram:

$P(\text{transforms} \mid \text{learning}) = C(\text{learning transforms}) / C(\text{learning})$

$C(\text{learning transforms}) = 0$

Therefore:

$P(\text{transforms} \mid \text{learning}) = 0 / 3 = 0$

Step 3: Back off to Unigram:

$P(\text{transforms}) = C(\text{transforms}) / \text{Total Words}$

$C(\text{transforms}) = 0$

Therefore:

$P(\text{transforms}) = 0$.

Final Backoff Probability:

$P(\text{transforms} \mid \text{machine learning}) = 0$.

The word "transforms" is not present anywhere in the training corpus, so even the unigram model cannot assign it a non-zero probability. In a practical system, an UNK or unknown-word token can be introduced to handle such words.

Importance of Backoff:

Backoff is useful because when a trigram is unseen, the model can use lower-order bigram or unigram information instead of immediately failing. This improves prediction coverage for sparse data and previously unseen query sequences.

Q3. Calculate the Deleted Interpolation probability of "machine learning improves".

Given:

$\lambda_1 = 0.5$ (Trigram)

$\lambda_2 = 0.3$ (Bigram)

$\lambda_3 = 0.2$ (Unigram)

Trigram probability:

$P(\text{improves} \mid \text{machine learning}) = 0.33$

Bigram probability:

$P(\text{improves} \mid \text{learning}) = C(\text{learning improves}) / C(\text{learning})$

$= 1 / 3$

$= 0.33$

Unigram probability:

Total words = 12

$C(\text{improves}) = 1$

$P(\text{improves}) = 1 / 12$

$= 0.0833$

Deleted Interpolation Formula:

$P = \lambda_1 P(\text{trigram}) + \lambda_2 P(\text{bigram}) + \lambda_3 P(\text{unigram})$

$P = (0.5 \times 0.33) + (0.3 \times 0.33) + (0.2 \times 0.0833)$

$P = 0.165 + 0.099 + 0.01666$

$P = 0.28066$

Therefore, the Deleted Interpolation probability is approximately:

$P(\text{machine learning improves}) = 0.281$

Interpretation:

Interpolation combines information from trigram, bigram, and unigram models. Therefore, even when one model has limited information, the lower-order models contribute to the final probability. This makes the prediction more robust when training data is sparse.

Q4. Calculate the Entropy of the prediction distribution.

Given:

$P(\text{improves}) = 0.33$

$P(\text{enables}) = 0.33$

$P(\text{drives}) = 0.33$

Formula:

$H = -\sum P(x) \log_2 P(x)$

$H = -[0.33 \log_2(0.33) + 0.33 \log_2(0.33) + 0.33 \log_2(0.33)]$

$H \approx 1.583$ bits

The maximum entropy for three equally likely outcomes is:

$H_{\text{max}} = \log_2(3) = 1.585$ bits

Therefore, the prediction distribution has almost maximum entropy.

Interpretation:

The three words "improves", "enables", and "drives" have almost equal probabilities. Therefore, the system has high uncertainty about which word will occur next. No single word is strongly preferred, so the search prediction system must consider all three possibilities.

Q5. Python Program

```
import nltk

import numpy as np

import math

from collections import Counter

corpus="machine learning improves business, machine learning enables automation,
machine learning drives innovation."

tokens=nltk.word_tokenize(corpus.lower())

unigrams=Counter(tokens)

bigrams=Counter(zip(tokens,tokens[1:]))

trigrams=Counter(zip(tokens,tokens[1:],tokens[2:]))

total=len(tokens)

print("Tokens:",tokens)

print("\nUnigram Counts:")

for k,v in unigrams.items(): print(k,":",v)

print("\nBigram Counts:")

for k,v in bigrams.items(): print(k,":",v)

print("\nTrigram Counts:")

for k,v in trigrams.items(): print(k,":",v)

p_bigram=bigrams[("machine","learning")]/unigrams["machine"]

print("\nMLE Bigram Probability:")

print("P(learning|machine)
=",bigrams[("machine","learning")],"/",unigrams["machine"],"=",p_bigram)

p_tri=trigrams[("machine","learning","transforms")]/bigrams[("machine","learning")]

p_bi=bigrams[("learning","transforms")]/unigrams["learning"] if unigrams["learning"]
else 0

p_uni=unigrams["transforms"]/total
```

```

print("\nBackoff Calculation:")
print("Trigram =",p_tri)
print("Bigram =",p_bi)
print("Unigram =",p_uni)
print("Backoff Probability =",p_uni)
p3=trigrams[("machine","learning","improves")]/bigrams[("machine","learning")]
p2=bigrams[("learning","improves")]/unigrams["learning"]
p1=unigrams["improves"]/total
l1,l2,l3=0.5,0.3,0.2
p_interp=l1*p3+l2*p2+l3*p1
print("\nDeleted Interpolation:")
print("Trigram =",p3)
print("Bigram =",p2)
print("Unigram =",p1)
print("Final Probability =",p_interp)
probs=np.array([0.33,0.33,0.33])
entropy=-sum(p*math.log2(p) for p in probs)
print("\nEntropy =",entropy,"bits")
print("\nFinal Next-Word Predictions:")
print("improves = 0.33")
print("enables = 0.33")
print("drives = 0.33")

```

Result

The e-commerce search system successfully uses MLE, Backoff, Deleted Interpolation, and Entropy Analysis. The MLE probability $P(\text{learning}|\text{machine})$ is 1.0. The unseen word "transforms" receives zero probability because it is absent from the corpus. Deleted Interpolation gives a probability of approximately 0.281 for "machine learning improves", while the prediction entropy is approximately 1.583 bits, indicating high uncertainty among the three possible next words.

CASE STUDY 2: AI-POWERED HOSPITAL APPOINTMENT CHATBOT

Q1. Assign Penn Treebank POS tags to both sentences.

Sentence 1:

"Book an appointment with the doctor."

Book/VB an/DT appointment/NN with/IN the/DT doctor/NN

Explanation:

"Book" is a Verb (VB) because it expresses an action or command. The sentence instructs the chatbot to perform the action of booking an appointment.

Sentence 2:

"The book contains medical information."

The/DT book/NN contains/VBZ medical/JJ information/NN

Explanation:

"Book" is a Noun (NN) because it refers to a physical or conceptual object. It is preceded by the determiner "The" and acts as the subject of the sentence.

Therefore:

Sentence 1 → book/VB

Sentence 2 → book/NN

The difference is caused by grammatical context.

Q2. Calculate the likelihood of "book" being VB and NN.

Given:

$$P(\text{book} \mid \text{VB}) = 0.7$$

$$P(\text{book} \mid \text{NN}) = 0.3$$

$$P(\text{Start} \rightarrow \text{VB}) = 0.6$$

$$P(\text{Start} \rightarrow \text{NN}) = 0.4$$

For VB:

$$P(\text{book}, \text{VB}) = P(\text{Start} \rightarrow \text{VB}) \times P(\text{book} \mid \text{VB})$$

$$= 0.6 \times 0.7$$

$$= 0.42$$

For NN:

$$P(\text{book}, \text{NN}) = P(\text{Start} \rightarrow \text{NN}) \times P(\text{book} \mid \text{NN})$$

$$= 0.4 \times 0.3$$

$$= 0.12$$

Therefore:

$$\text{VB likelihood} = 0.42$$

$$\text{NN likelihood} = 0.12$$

Normalized probabilities:

$$P(\text{VB} \mid \text{book}) = 0.42 / (0.42 + 0.12)$$

$$= 0.7778$$

$$P(\text{NN} \mid \text{book}) = 0.12 / (0.42 + 0.12)$$

$$= 0.2222$$

Final Result:

VB is the most probable tag at the beginning of a sentence.

The model favors VB because both the emission probability $P(\text{book} \mid \text{VB})$ and the start transition probability $P(\text{Start} \rightarrow \text{VB})$ are greater than their corresponding NN probabilities.

Note: This calculation uses only the supplied start and emission probabilities. A complete HMM would also use transition probabilities between subsequent tags, which would allow the context of the second sentence to favor NN.

Q3. Compare Rule-Based and Stochastic HMM POS Tagging.

Rule-Based POS Tagging:

Rule-based tagging uses manually designed grammatical rules and dictionaries. For example, "book" can be classified as a verb when it appears in a command such as "Book an appointment", while it can be classified as a noun after a determiner such as "the book".

Advantages:

1. Easy to understand and interpret.
2. Does not require a large training corpus.
3. Rules can be specifically designed for healthcare terminology.
4. Easy to apply known grammatical patterns.

Limitations:

5. Requires many manually written rules.
6. Difficult to handle exceptions and complex contexts.

7. Rules may fail for ambiguous sentences.

Stochastic HMM Tagging:

HMM tagging uses emission probabilities and tag transition probabilities learned from a tagged corpus.

Advantages:

8. Handles contextual ambiguity using probabilities.
9. Learns patterns from training data.
10. Can adapt better to different sentence structures.
11. Suitable for large-scale chatbot applications.

Limitations:

12. Requires training data.
13. Performance depends on corpus quality.
14. Rare and unseen words may cause difficulties.

For a healthcare chatbot, a combination of both methods is useful because grammatical rules can provide reliable domain-specific behavior while HMM probabilities can resolve contextual ambiguity.

Q4. Importance of the Penn Treebank Tagset in Healthcare Chatbots.

The Penn Treebank Tagset provides standardized grammatical labels for words. These tags help downstream NLP components understand sentence structure.

Intent Detection:

POS tags help distinguish actions from objects. For example, "Book an appointment" contains a verb "Book", indicating an appointment-booking intent.

Entity Identification:

Noun tags help identify important entities such as doctor, appointment, medicine, hospital, and patient.

Response Generation:

POS information helps the chatbot understand grammatical relationships and construct meaningful responses.

Ambiguity Resolution:

POS tags help distinguish words such as "book", which can act as a noun or verb depending on context.

Overall, standardized POS tags improve consistency between different NLP components and support accurate intent detection, entity identification, and response generation.

Q5. Python Program

```
import nltk
```

```
sentences=["Book an appointment with the doctor.,"The book contains medical information."]
```

```
for sentence in sentences:
```

```
    tokens=nltk.word_tokenize(sentence)
```

```

print("\nSentence:",sentence)

print("Tokens:",tokens)

print("POS Tags:",nltk.pos_tag(tokens))

p_book_vb=0.7

p_book_nn=0.3

p_start_vb=0.6

p_start_nn=0.4

likelihood_vb=p_start_vb*p_book_vb

likelihood_nn=p_start_nn*p_book_nn

total=likelihood_vb+likelihood_nn

posterior_vb=likelihood_vb/total

posterior_nn=likelihood_nn/total

print("\nHMM Calculation:")

print("VB =",p_start_vb,"*",p_book_vb,"=",likelihood_vb)

print("NN =",p_start_nn,"*",p_book_nn,"=",likelihood_nn)

print("P(VB|book) =",posterior_vb)

print("P(NN|book) =",posterior_nn)

print("\nMost Probable Tag:", "VB" if likelihood_vb>likelihood_nn else "NN")

```

Sample Output

Sentence: Book an appointment with the doctor.

Tokens: ['Book','an','appointment','with','the','doctor','.']

POS	Tags:
[('Book','VB'),('an','DT'),('appointment','NN'),('with','IN'),('the','DT'),('doctor','NN'),('.',	
')]]	

Sentence: The book contains medical information.

Tokens: ['The','book','contains','medical','information','.']

POS Tags:
[('The','DT'),('book','NN'),('contains','VBZ'),('medical','JJ'),('information','NN'),('.', '.')]]

HMM Calculation:

$$VB = 0.6 * 0.7 = 0.42$$

$$NN = 0.4 * 0.3 = 0.12$$

$$P(VB|book) = 0.7777777778$$

$$P(NN|book) = 0.2222222222$$

Most Probable Tag: VB

Result

The chatbot successfully identifies the contextual POS of "book". In "Book an appointment", it acts as a verb, while in "The book contains medical information", it acts as a noun. The HMM calculation gives a higher probability to VB at the beginning of a sentence based on the given probabilities.

CASE STUDY 3: FINANCIAL NEWS POS TAG CORRECTION AND UNCERTAINTY ANALYSIS

Q1. Apply the Transformation Rule.

Initial POS sequence:

market/NN growth/NN drives/NNS investment/NN

Rule:

Change NNS to VBZ if the preceding word is tagged as NN.

For "drives":

Previous word = growth

growth is tagged as NN.

Therefore:

drives/NNS → drives/VBZ

Corrected POS sequence:

market/NN growth/NN drives/VBZ investment/NN

The transformation is linguistically appropriate because "drives" functions as the main verb of the sentence. The singular subject "growth" agrees with the singular third-person verb "drives".

Q2. Analyze the tagging error for "drives".

Initial tag:

drives/NNS

NNS means plural noun.

However, in the sentence:

"Market growth drives investment."

"Market growth" is the subject and "drives" expresses the action performed by the subject.

Therefore, "drives" is a third-person singular present-tense verb and should be tagged VBZ.

Correct tagging:

Market/NN growth/NN drives/VBZ investment/NN

The surrounding grammatical context helps determine the correct tag because "drives" occurs after the noun phrase "market growth" and before the noun "investment". The sentence structure indicates that "drives" is functioning as the main verb.

Q3. Calculate the relative frequency/probability.

Given:

market = 500

growth = 350

drives = 180

investment = 420

Total frequency:

$500 + 350 + 180 + 420 = 1450$

Formula:

$P(\text{word}) = \text{Frequency}(\text{word}) / \text{Total Frequency}$

For market:

$P(\text{market}) = 500 / 1450 = 0.3448$

For growth:

$P(\text{growth}) = 350 / 1450 = 0.2414$

For drives:

$P(\text{drives}) = 180 / 1450 = 0.1241$

For investment:

$P(\text{investment}) = 420 / 1450 = 0.2897$

Table:

Word	Frequency	Relative Probability
market	500	0.3448

growth	350	0.2414
drives	180	0.1241
investment	420	0.2897

The probabilities sum approximately to 1.

Corpus frequency information supports probabilistic NLP by showing how frequently words occur. Frequent words generally receive higher probabilities and can contribute to statistical POS tagging, language modeling, prediction, and ambiguity resolution.

Q4. Calculate entropy before and after applying the transformation rule.

Probability distribution:

market = 0.3448

growth = 0.2414

drives = 0.1241

investment = 0.2897

Formula:

$$H = -\sum P(x) \log_2 P(x)$$

Therefore:

$$H = -[(0.3448 \log_2 0.3448) + (0.2414 \log_2 0.2414) + (0.1241 \log_2 0.1241) + (0.2897 \log_2 0.2897)]$$

$$H \approx 1.916 \text{ bits}$$

Before transformation:

$$H = 1.916 \text{ bits}$$

The transformation rule changes the POS tag of "drives" from NNS to VBZ, but it does not change the word frequencies.

After transformation:

$$H = 1.916 \text{ bits}$$

Therefore:

$$\text{Entropy before transformation} = 1.916 \text{ bits}$$

$$\text{Entropy after transformation} = 1.916 \text{ bits}$$

Interpretation:

The entropy remains unchanged because entropy was calculated from the word-frequency distribution, and the transformation changes only the POS tag, not the word frequencies. Thus, the transformation improves tagging correctness without changing the uncertainty of the word-frequency distribution.

If entropy were instead calculated over the POS-tag distribution, then the tag transformation could change the entropy. However, using the word-frequency statistics specified in the case study, the entropy remains the same.

Q5. Python Program

```
import nltk
```

```
import math
```

```
from collections import Counter
```

```
sentence="Market growth drives investment."
```

```
tokens=nltk.word_tokenize(sentence.lower())
```

```
initial_tags=[("market","NN"),("growth","NN"),("drives","NNS"),("investment","NN")]
```

```
print("Tokens:",tokens)
```

```

print("\nInitial POS Tags:")
for word,tag in initial_tags: print(word+"/"+tag,end=" ")
corrected=initial_tags.copy()
for i in range(1,len(corrected)):
    if corrected[i][1]=="NNS" and corrected[i-1][1]=="NN":
        corrected[i]=(corrected[i][0],"VBZ")
print("\n\nCorrected POS Tags:")
for word,tag in corrected: print(word+"/"+tag,end=" ")
freq={"market":500,"growth":350,"drives":180,"investment":420}
total=sum(freq.values())
probs={word:count/total for word,count in freq.items()}
print("\n\nFrequency Distribution:")
for word,p in probs.items(): print(word,":",freq[word],"->",p)
entropy=-sum(p*math.log2(p) for p in probs.values())
print("\nEntropy Before Transformation:",entropy)
print("Entropy After Transformation:",entropy)
print("\nFinal POS Tagging:")
print(" ".join(word+"/"+tag for word,tag in corrected))

```

Sample Output

Tokens: ['market','growth','drives','investment','.']

Initial POS Tags:

market/NN growth/NN drives/NNS investment/NN

Corrected POS Tags:

market/NN growth/NN drives/VBZ investment/NN

Frequency Distribution:

market : 500 -> 0.3448275862

growth : 350 -> 0.2413793103

drives : 180 -> 0.1241379310

investment : 420 -> 0.2896551724

Entropy Before Transformation: 1.9160904909

Entropy After Transformation: 1.9160904909

Final POS Tagging:

market/NN growth/NN drives/VBZ investment/NN

Result

The Transformation-Based Tagger successfully corrected "drives/NNS" to "drives/VBZ" using the rule that changes NNS to VBZ when the preceding word is tagged NN. The corpus probabilities were calculated from the given frequencies, and the entropy of the word-frequency distribution was found to be approximately 1.916 bits both before and after transformation.